

LLD Case Study 16: Kafka Consumer Group Partition Rebalancing

The Bridge: How Do Consumers Share Work at Scale? In Chapter 16 of Track 1, we learned that Apache Kafka scales horizontally by splitting a topic into **Partitions** (e.g. 12 partitions).

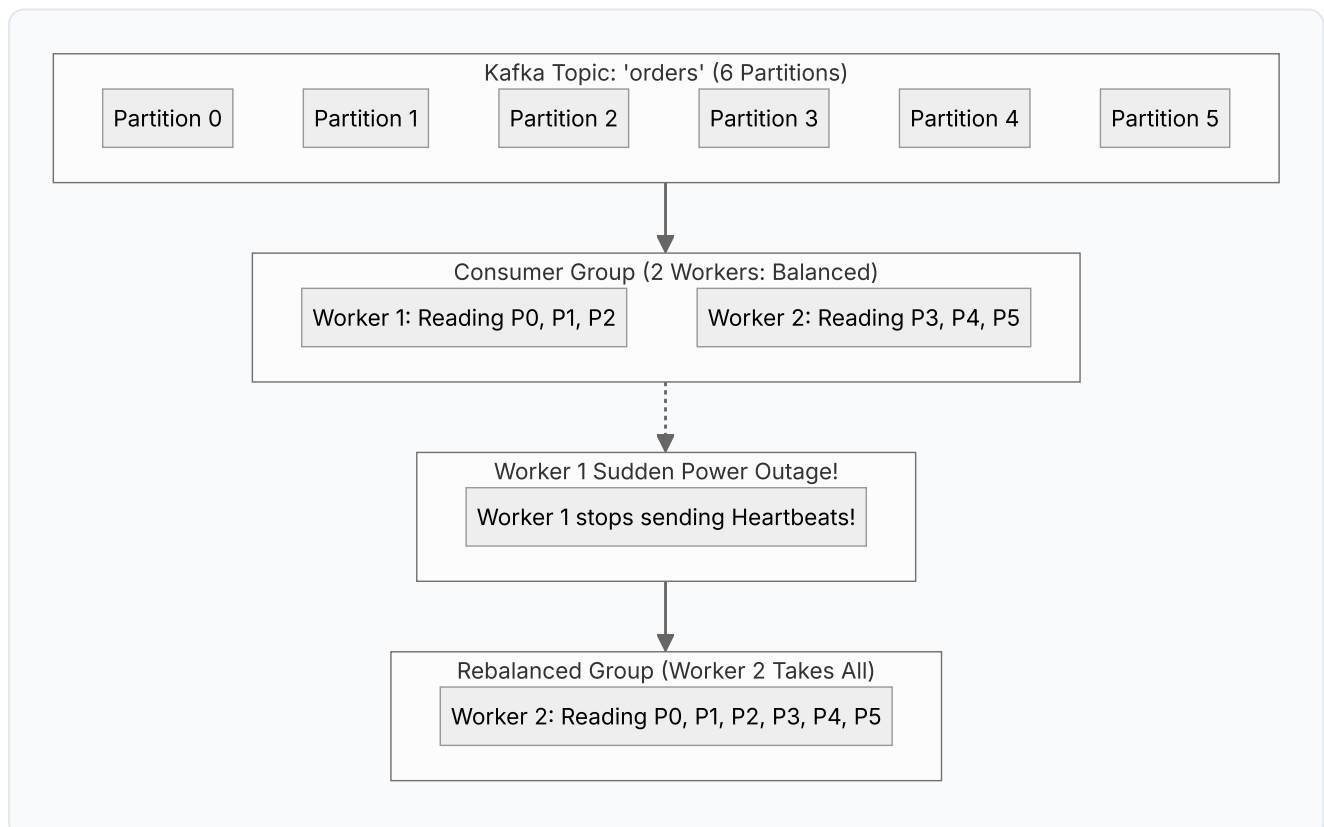
But what happens inside a **Consumer Group** when you have 4 worker applications reading those 12 partitions, and suddenly:

1. Worker 3 crashes!
2. Or you deploy 2 new servers to handle a traffic surge!

How do the surviving workers detect the crash, renegotiate ownership of the 12 partitions, and resume reading without duplicating or dropping messages?

This process is called **Consumer Group Partition Rebalancing**. In this case study, we will design the industrial **Cooperative Sticky Assignor** protocol and examine how modern streaming brokers avoid catastrophic "Stop-The-World" rebalance storms.

1. Zero-Prerequisite Intuition: The "Airport Baggage Carousel Shift"



The Eager vs Cooperative Rebalance Revolution

- **The Legacy Eager Way ("Evasive Stop-The-World"):** When Worker 1 died, Kafka forced ALL surviving workers to drop ALL their assigned partitions. The entire message processing pipeline stopped cold for 30 seconds while everyone renegotiated from zero!
 - **The Modern Cooperative Sticky Way (Incremental Handover):**
 - Surviving workers keep processing their existing partitions without interruption.
 - Only the orphaned partitions (from the dead worker) are reassigned to the surviving workers.
 - Downtime drops from 30 seconds to **0 seconds!**
-

2. Requirements & Architecture

Functional Requirements:

1. **Group Membership & Heartbeats:** Workers periodically send heartbeats to the Group Coordinator. If no heartbeat arrives within `session_timeout_ms`, the worker is marked dead.
 2. **Deterministic Partition Assignment:** Distribute M partitions across N active consumers fairly and evenly.
 3. **Sticky Allocation:** When rebalancing occurs, minimize partition movement (keep partitions on their previous consumers if still alive).
 4. **Generation ID (Epoch Fencing):** Every rebalance increments a `generation_id` to reject stale commits from zombie workers.
-

3. Production-Grade Python Implementation



4. Milestone Check: What You Just Mastered!

You now understand:

1. How Kafka divides topic partitions among consumer group members.
2. Why **Stop-The-World** rebalances caused multi-minute production outages in legacy systems.
3. How the **Cooperative Sticky Assignor** preserves live partition assignments while transferring only orphaned partitions.
4. How **Generation IDs** prevent stale writes from zombie workers that temporarily disconnected.

